

Vossie Marketplace Knowledge Base

Vossie Marketplace — All-in-One Context

This file concatenates the curated Markdown knowledge base for easy upload to a fresh ChatGPT account. The categorized files remain preferable for durable project use, but this file is useful when only one context file can be supplied.

SOURCE FILE: 00__START__HERE/01__PROJECT__CONTEXT.md

Project Context

Origin

Vossie began as an Eduvos student marketplace concept and evolved through the EduHackers research/build process into a much broader campus-commerce operating system.

The project combines:

- verified student identity,
- local campus buying and selling,
- student courier participation,
- campus hub custody and collection,
- cross-campus parcel movement,
- seller analytics and incubation support,
- operations/admin tooling,
- payments, disputes and auditability,
- short-form commerce discovery through Marketplace Reels,
- buyer-side Requests when a needed product/service is not listed.

Core differentiation

The strongest thesis is not “marketplace for students.” The stronger combination is:

verified campus identity + multi-role student participation + campus logistics/custody + cross-campus transfers + operations tooling + demand intelligence for student entrepreneurship.

Current project state

The work now exists across strategy PDFs, a 177-screen UI/UX atlas, developer guides, motion specifications, business research, marketing materials and a cinematic pitch blueprint.

The project should be treated as a system with a canonical hierarchy of decisions. Older prototypes and older PDFs are useful references, but newer confirmed decisions override them.

Prototype philosophy

The demo should feel real and interactive:

- real state transitions inside the prototype,
- simulated/sandbox external integrations,
- real cross-device responsive behaviour,
- no false claims of live Eduvos, payment-provider or courier-provider connectivity,

- deterministic demo data and reset paths,
- polished motion and microinteractions.

Zero-budget principle

Use free tiers, local simulation and open/free tools for the prototype. Keep all provider-specific integrations behind adapters so the implementation can switch later without rebuilding the product domain.

SOURCE FILE: 00__START__HERE/02__MASTER__CONTEXT__TRANSFER__I

Master Context Transfer Prompt

Paste this into a fresh ChatGPT account after uploading this Markdown knowledge base and any canonical PDFs/Figma links:

You are now the lead product, design, engineering and research copilot for Vossie Marketplace / EduHackers 2.0. Read the entire uploaded Vossie Markdown knowledge base before taking action. Treat the Markdown pack as curated context and the latest explicitly approved brand/UI/engineering files as source-of-truth assets. Do not assume older prototypes override newer confirmed decisions. Vossie is a two-application platform: App 1 Marketplace for Buyer/Seller/Courier under one verified student identity, and App 2 Operations for Campus Hub, Transfers, Incubation, Support, Moderation, Finance, Compliance, Security, Admin and Platform Operations. Preserve the zero-budget prototype constraint. Do not pretend institutional SSO, payments, third-party logistics or production credentials are live when they are not. Use real tools only when connected and available. Keep architecture, Figma, code, GitHub and pitch assets synchronized through explicit handoffs rather than hidden chat memory. Before changing the product, read 00__START__HERE/04__LOCKED__DECISIONS.md and 00__START__HERE/05__OPEN__DECISIONS.md.

For a Figma-focused session, add:

Use Figma as the UI/UX source of truth. Build variables/tokens, components, variants, Auto Layout, responsive states and motion contracts before scaling into the 177-screen families. Use the Plato-inspired softness and Uber-inspired interaction clarity described in the design files without copying proprietary assets.

For engineering, add:

Follow Developer Series 1 and 3: TypeScript/Next.js/Supabase foundation, GitHub as durable source of truth, Cursor build agents led by Atlas, VS Code verification led by Sentinel, and gated CI/CD. Do not push directly to main or waive failed security/data gates.

SOURCE FILE: 00__START__HERE/03__SOURCE__OF__TRUTH__ORDER.md

Source of Truth Order

When documents disagree, use this precedence:

1. **Latest explicitly confirmed user decision** in the curated Markdown pack.
2. **Approved Vossie Brand Identity** for logo, colour, typography and visual language.
3. **Latest Platform Strategy / System Blueprint (V12)** for product policy and system model.
4. **Full UI/UX Page Architecture / 177-screen atlas** for screen inventory and role flows.
5. **Developer Series 1** for stack and application architecture.
6. **Developer Series 2** for Figma and runtime motion production.
7. **Developer Series 3** for agentic SDLC and GitHub workflow.
8. **Developer Series 4** for cinematic pitch production.
9. Older prototypes and earlier blueprint versions as historical references only.

Conflict rule

Do not silently merge contradictory rules. Record the conflict, identify which source is newer/higher priority, and resolve explicitly.

Prototype vs production

A specification can describe production intent without proving production readiness. Always distinguish:

- specified,
 - prototyped/simulated,
 - locally implemented/tested,
 - externally integrated,
 - production approved.
-

SOURCE FILE: 00__START__HERE/04__LOCKED__DECISIONS.md

Locked Decisions

The following decisions are considered current unless the user explicitly changes them.

Product

- One platform, two application families.
- One student identity; Buyer is base capability, Seller/Courier are progressive activations.
- App 1 = Buyer + Seller + Courier.
- App 2 = Campus Hub + Transfers + Incubation + operational/admin workspaces.
- Marketplace Reels is a first-class commerce feature, not a generic social network.
- Buyer Requests exist alongside normal Listings.
- Cross-campus uses hub-network routing with third-party courier fallback.

Money

- Pilot seller commission hypothesis: **7%**.
- Pilot courier platform commission hypothesis: **5%** of courier delivery earnings.
- No separate buyer percentage commission; buyer pays applicable delivery by default.
- Earnings lifecycle: **Pending → Secured → Available → Payout Processing → Paid Out**.
- Seller/courier may choose automatic payout or manual withdrawal.
- Package QR proves custody; recipient OTP proves final delivery.
- OTP delivery starts a **24-hour issue window**.
- Full approved refunds reverse Vossie seller commission; provider costs remain separately tracked.

Authentication

- EDUV student-number format or email domain alone is not authentication.
- Production target: institutional SSO/federation with Eduvos/Microsoft/Entra and existing Authenticator MFA.
- Vossie must not store institutional passwords or Authenticator secrets.
- Prototype uses a clearly labelled simulated/sandbox version of the target flow.
- No requirement for separate Vossie Face ID/passkey layer in the current prototype.

Design

- Plato inspires visual softness; Uber inspires interaction architecture.
- Vossie brand identity remains authoritative.
- Mobile-first Marketplace; desktop-first Operations; all surfaces responsive.
- Motion must explain state and causality; reduced-motion support required.

Engineering

- One adaptive PWA URL/QR for the prototype.
 - TypeScript + Next.js/React + PostgreSQL/Supabase foundation.
 - GitHub monorepo is durable source of truth.
 - Cursor = implementation/build cockpit led by Atlas.
 - VS Code = independent verification/control cockpit led by Sentinel.
 - No autonomous direct-to-main pushes.
-

SOURCE FILE: 00_START_HERE/05_OPEN_DECISIONS.md

Open Decisions and External Dependencies

These areas are intentionally not fully locked.

Institutional

- Formal Eduvos approval for SSO/identity federation.
- Exact active-student eligibility API/attribute source.
- Campus-hub operational agreements and staffing.
- Institutional governance, POPIA and procurement requirements.

Payments

- Final production PSP selection.
- Exact marketplace split/payout capability and pricing.
- Merchant onboarding and settlement terms.
- Treatment of provider fees in refunds and low-value orders.

Logistics

- Final third-party courier partners/API support.
- Real cross-campus route schedules, capacity and economics.
- Whether transfer couriers are students, institution transport, third parties, or a hybrid.

Technology

- Final identity provider implementation (Clerk is a candidate; institutional SSO remains the ultimate target).
- Final push/email provider.
- Final observability provider.
- Final production video/media pipeline.
- Native iOS/SwiftUI expansion only if real native requirements justify it.

Legal/compliance

- Food-selling requirements by relevant municipality/campus context.
 - Courier insurance/legal requirements.
 - Digital-content ownership and academic-integrity policy.
-

Product Vision and Differentiation

Vision

Vossie is a campus-commerce operating system for verified students. It is designed to connect everyday student demand with student supply, local fulfilment, accountable custody and entrepreneurship support.

Differentiation stack

The defensible idea is the combination of:

1. verified student identity,
2. Buyer/Seller/Courier participation in one account,
3. campus-local marketplace density,
4. Request marketplace for unmet demand,
5. Marketplace Reels for commerce discovery,
6. QR/OTP custody and delivery proof,
7. Campus Hubs and cross-campus transfers,
8. Incubation intelligence from real demand/supply signals,
9. Operations tooling for support, disputes, finance, compliance and security.

Competitive framing

Large e-commerce marketplaces have selection and mature fulfilment but do not optimize for verified campus peer commerce. Social/DM commerce has discovery but weak transaction state and accountability. Classifieds have broad supply but not integrated campus identity, custody, courier and incubation. On-demand delivery has strong logistics but is usually merchant-led rather than peer campus commerce.

Product thesis

Vossie should be pitched as **campus commerce infrastructure**, not another classifieds board.

Two-App Architecture and Roles

App 1 — Vossie Marketplace

Student-facing application. One identity can hold multiple capabilities.

Buyer

- browse/search,
- categories,
- listing detail,
- seller storefronts,
- Reels,
- cart/checkout,
- Requests,
- messaging,
- order tracking,
- pickup/delivery,
- disputes/refunds,
- ratings/reviews.

Seller

Day-to-day commerce mode:

- listings,
- incoming orders,
- Requests/offers,
- inventory/availability,
- quick analytics,
- earnings/payout state,
- customer/order actions,
- Reels publishing.

Courier

- activation/verification,
- go online/offline,
- eligible jobs,
- map/route,
- pickup,
- QR custody scans,
- OTP delivery,
- incidents,
- earnings.

App 2 — Vossie Operations

One staff/business application with permission-controlled workspaces.

Campus Hub

Intake, condition check, QR custody, pickup/release, outgoing transfers, returns, exceptions.

Transfers

Routes, manifests, hub-to-hub scans, reconciliation, mismatch/incident handling.

Incubation

Demand intelligence, seller health, unmet searches, trends, interventions and business-growth plans.

Operations/Admin

Support, moderation, disputes, finance, compliance, security, audit, users/roles, configuration and platform health.

Permission principle

A visible button is not authority. Backend/server policy is authoritative. Roles must be scoped to the user, campus and resource.

SOURCE FILE: 01_PRODUCT/03_CORE_USER_JOURNEYS.md

Core User Journeys

Buyer vertical slice

Continue with Eduvos → verified student session → Marketplace Home → Search/Reel → Listing → Cart → Checkout → PSP sandbox → Payment Secured → seller fulfils → courier/hub tracking → QR/OTP handover → Delivered → 24-hour issue window → Completed.

Buyer Request

No suitable result → Request It → choose category, describe need, campus/time/budget, optional media → matching → sellers receive request cards → sellers send offers → buyer accepts one → normal order/booking flow begins.

Seller activation

Verified Buyer → Activate Seller → identity/onboarding → seller profile/store setup → payout settings → listing creation → publish → order/request flow.

Courier activation

Verified student → Activate Courier → select walking/bicycle/motorcycle/car → provide role-specific evidence → approval → online state → job → pickup → scan → route → OTP handover → earnings.

Campus Hub

Package arrives → scan QR → condition check → custody state changes → ready for pickup/courier/transfer → scan out → reconciliation/audit.

Cross-campus

Origin seller/hub → route through nearest viable campus hubs or approved third-party courier → each custody leg scanned → buyer sees one shipment timeline → final local handover.

SOURCE FILE: 01_PRODUCT/04_SCREEN_ARCHITECTURE_177.md

177-Screen Architecture Summary

Current full design inventory:

Workspace	Screens	Scope
Access / Authentication	12	registration, verification, recovery, staff entry
Shared / Account	24	account, role switching, messages, support, wallet/security
Buyer	41	home, explore, activity, orders, requests
Seller	25	requests, shop, earnings, account
Courier	14	offers, delivery, earnings, account
Campus Hub	8	intake, scans, transfers, account
Transfers	9	routes, manifests, scans, issues
Incubation	12	overview, demand, businesses, plans
Operations	17	moderation, finance, support, compliance, security, admin
Extensions	15	feature-gated categories/workspaces
Total	177	

Application family totals

- App 1 role-specific: Buyer 41 + Seller 25 + Courier 14 = 80, with Access/Shared layered across them.
- App 2 operational: Hub 8 + Transfers 9 + Incubation 12 + Operations 17 = 46.

Important

The 177-screen universe is the architecture, not the first MVP commitment. Build complete vertical journeys first.

SOURCE FILE: 01_PRODUCT/05_REQUEST_MARKETPLACE.md

Buyer Request Marketplace

Buyer Requests sit beside ordinary Listings.

Buyer flow

1. Tap **Request**.
2. Select category/subcategory.
3. Describe need.
4. Add campus, desired time, budget and optional photo/file.
5. Submit.
6. System matches eligible sellers.
7. Sellers receive Uber-like request cards.
8. Sellers respond with structured offers: price, timing, terms.
9. Buyer compares offers and accepts one.
10. Accepted offer becomes the standard order/booking flow.

Design principles

- Avoid chaotic bidding wars.
- Sellers should know request scope before accepting.
- Requests expire/close cleanly.
- Abuse/reporting/moderation applies.
- Incubation receives aggregate unmet-demand signals.

Demo value

Request → seller notification → offer → buyer acceptance → sandbox checkout is one of the strongest demo stories because it proves Vossie creates supply from demand, not only displays existing stock.

SOURCE FILE: 01_PRODUCT/06_MARKETPLACE_REELS.md

Marketplace Reels

Marketplace Reels is a shoppable short-video layer tied to real commerce objects.

Buyer experience

- first-class Reels tab in Marketplace navigation,
- vertical swipe feed,
- seller handle/profile,
- short caption,
- linked product/service/rental/digital item,
- pinned product rail,
- multiple linked listings where appropriate,
- tap through to product/store/cart without leaving commerce context.

Seller experience

Create Reel → record/upload → trim to roughly 15–30 seconds → caption → link listing(s) → preview → publish.

Seller analytics should include views, product taps, add-to-cart, bookings/orders and conversion signals.

Boundaries

- no requirement for open comments in the pilot,
 - content must remain commerce-led,
 - moderation/reporting required,
 - a Reel must link to legitimate marketplace inventory/service content.
-

SOURCE FILE: 02__IDENTITY__SECURITY/01__AUTHENTICATION__EDUV

Authentication, Eduvos Identity and MFA

Security principle

Identifier format is not proof of identity. An EDUV-prefixed student number or institutional-looking email may be validated syntactically, but neither grants access by itself.

Production target

Vossie → **Continue with Eduvos** → Eduvos/Microsoft identity provider → existing Microsoft Authenticator/MFA → signed identity response → Vossie validates issuer/tenant/token → Vossie verifies active-student eligibility → Vossie session.

Vossie should not receive/store:

- institutional password,
- Authenticator seed,
- MFA approval code.

Separate concepts

- **Authentication:** control of institutional identity.
- **Eligibility:** active student allowed to participate.
- **Authorization:** Buyer/Seller/Courier/staff permissions inside Vossie.

Prototype

Use a clearly marked sandbox simulation:

Continue with Eduvos → demo Microsoft-style sign-in → simulated Authenticator approval → identity verified → campus/student status displayed → Vossie session.

Never claim the demo is connected to Eduvos production infrastructure.

SOURCE FILE: 02__IDENTITY__SECURITY/02__ROLE__REGISTRATION__VF

Role Registration and Verification

One identity, progressive capabilities

There are not three separate login systems.

Buyer

Base verified student capability after institutional identity/eligibility.

Seller activation

Buyer verification + seller onboarding + agreements + store/business setup + payout information + review/approval where required.

Courier activation

Reuse verified identity. Add transport mode and courier-specific evidence.

Transport options:

- walking,
- bicycle,
- motorcycle,
- car.

Walking/bicycle should have lighter onboarding and geographic/package constraints. Motorcycle/car require licence/vehicle evidence appropriate to the production policy.

Staff roles

Hub, Finance, Security, Admin, Incubation etc. are assigned memberships, not self-selected public roles.

Role reuse

If identity verification was already completed for Seller and the same user later activates Courier, reuse what is still valid instead of collecting duplicate sensitive information.

SOURCE FILE: 02_IDENTITY_SECURITY/03_RBAC_PRIVACY_AUDIT.n

RBAC, Privacy, Audit and Security Operations

Authorization

Use capability-based RBAC plus resource/campus scoping. Client-side hiding is convenience only; server-side checks are authoritative.

Examples:

- Hub Operator cannot release payouts.
- Incubation cannot view security evidence unless granted.
- Finance cannot silently change user roles.
- Security/admin actions require strong auditability and, where appropriate, reauthentication/dual control.

Audit model

Keep security audit events separate from product analytics.

Audit examples:

- role change,
- payout-detail change,
- refund approval,
- evidence view,
- moderation action,
- login anomaly,
- admin configuration change.

Every sensitive action should be traceable UI → API/command → database/audit row.

Data minimization

- avoid long-term student-card image retention unless necessary,
 - do not expose private buyer/seller details to transfer couriers,
 - separate logs from secrets,
 - restrict evidence access,
 - retain only what operations/legal requirements justify.
-

SOURCE FILE: 03_COMMERCE/01_MONEY_MODEL_LEDGER.md

Pilot Money Model and Ledger

Pilot commercial hypotheses

- Seller commission: **7%** of seller transaction value.
- Courier commission: **5%** of courier delivery earnings.
- No listing fee or seller subscription in the pilot.
- No separate Vossie buyer percentage fee.
- Buyer pays applicable delivery by default.

These are hypotheses subject to PSP economics, legal/accounting review and pilot evidence.

Earnings states

Pending → Secured → Available → Payout Processing → Paid Out

Suggested meaning:

- Pending: order/earning exists but payment state not yet secured.
- Secured: PSP-confirmed payment exists; not yet releasable.
- Available: release conditions met.
- Payout Processing: payout command sent/processing.
- Paid Out: provider-confirmed settlement.

Payout preference

Seller/courier chooses:

- Automatic payout, or
- Manual withdrawal.

Only Available funds may move.

Ledger principle

Do not edit a single balance blindly. Use append-only/reference-based ledger entries with idempotency keys and provider references.

SOURCE FILE: 03_COMMERCE/02_PAYMENTS_GATEWAYS_SECURITY

Payments, Gateways and Security

Architecture

Buyer → PSP-hosted/tokenized checkout → provider server confirmation/webhook → Vossie verifies signature/idempotency → ledger entry/state transition → protection window → payout.

Vossie should store

- internal payment ID,
- provider reference,
- amount/currency,
- fee snapshot,
- status,
- seller/courier allocation,
- refund/payout references.

Vossie should not store

- raw card number,
- CVV,
- online-banking credentials.

Prototype methods to demonstrate

- card,
- Instant EFT / Pay by Bank,
- PayShap,
- Apple Pay / Google Pay where provider/device appropriate,
- sandbox only unless real credentials are provided.

Security controls

- backend-only secret keys,
- signed webhook verification,
- idempotent event processing,
- payout-detail changes protected with step-up security,
- reconciliation jobs,
- append-only financial history,
- client never declares final payment success by itself.

Provider strategy

Use a PSP adapter. Candidate providers were researched across Paystack, Payfast, Peach, Stitch/Ozow and others. Final production choice remains open.

SOURCE FILE: 03__COMMERCE/03__DISPUTES__REFUNDS__RETURNS.md

Disputes, Refunds and Returns

Delivery vs satisfaction

Recipient OTP proves **delivery**, not product satisfaction.

After OTP-confirmed delivery, the buyer receives a **24-hour issue window**.

Dispute flow

1. Buyer opens issue within allowed window.
2. Structured evidence is attached to the order case.
3. Affected seller funds are paused/frozen as appropriate.
4. Seller gets first opportunity to resolve within defined response policy.
5. Unresolved case escalates to Operations.
6. Operations reviews order data, chat/evidence, QR custody, OTP, payment and timeline.
7. Refund/partial refund/denial/return outcome is recorded and processed through the PSP adapter.

Return-cost responsibility

- Seller fault → seller bears return cost.
- Buyer change of mind / buyer fault where returns are allowed → buyer bears return cost.
- Courier/hub fault → logistics incident; buyer/seller should not automatically bear the cost.
- Unclear case → Operations adjudicates.

Courier earnings

A later product-quality dispute should not automatically freeze courier earnings when OTP-confirmed delivery was valid and courier service was not the cause.

SOURCE FILE: 03__COMMERCE/04__INVENTORY__TAXONOMY__LISTINGS

Inventory, Taxonomy, Listings and Promotions

Top-level category direction

Keep top-level navigation simple; use subcategories and dynamic attributes underneath.

Current structure:

- Food & Drinks
- Essentials
- Fashion & Lifestyle
- Services
- Rentals
- Digital
- future/feature-gated categories such as Events/Tickets if later approved

Dynamic listing model

One core listing model + category-specific extensions.

Examples:

- Food: ingredients, allergens, size, add-ons, prep time.
- Fashion: size, colour, variant.
- Services: slots/availability, duration, location/remote.
- Rentals: duration, deposit, availability, condition.
- Digital: file/access entitlement, ownership declaration, delivery rules.

Seller UI

Sellers fill structured fields; Vossie composes consistent cards and product pages. Sellers do not freely design marketplace cards.

Buyer card

Image → title → seller/trust context → price → special if valid → campus/fulfilment signals → CTA.

Promotions

Use structured fields for normal price, promotional price, dates and savings. Avoid fake “was” pricing.

Seller live preview

Listing editor should show how the buyer-facing card/detail page will look while the seller edits.

SOURCE FILE: 03_COMMERCE/05_FOOD_COMPLIANCE.md

Food Marketplace Compliance and Trust

Any eligible seller may use the Food category; Vossie does not create a completely separate food-seller identity.

Do not use “buy at your own risk” as the product message

That weakens trust and does not replace applicable obligations.

Product approach

Food listings should require stronger structured information and traceability:

- ingredients,
- allergens,
- prep/availability time,
- seller identity,
- food-specific reviews,
- order/batch/preparation timestamps where useful,
- clear report-food-safety flow.

Seller responsibility

Vossie provides platform rules, guidance, traceability and incident handling. Sellers remain responsible for meeting applicable food laws and campus requirements.

Incident flow

Food safety report → evidence/order linkage → urgent Operations review → affected listing/batch/seller may be paused → trace related orders → escalate according to policy.

Production dependency

Actual compliance requirements must be validated with Eduvos and relevant municipal/environmental-health authorities. Do not hard-code a broad “student exemption.”

SOURCE FILE: 03_COMMERCE/06_RECEIPTS_NOTIFICATIONS_EMAIL

Receipts, Notifications and Email System

Use one event-driven communications system.

Channels

- in-app notification centre,
- push notifications,
- email for important transactional/financial milestones,
- optional marketing communications separately consented/configured.

Notification Centre

Suggested tabs/categories:

- Orders,
- Payments,
- Messages,
- Marketplace,
- Account/Security.

Each notification deep-links to the actual object.

Transactional milestones

Examples:

- order accepted,
- payment secured,
- seller preparing,
- ready for pickup,
- courier assigned,
- hub received,
- transfer departed/arrived,
- delivered,
- refund approved,
- payout processed,
- security/account status.

Receipts

Permanent in Orders, view/download/share/email. Derived from order and ledger data, not manually typed text.

Operations monitor

Track communication event, recipient, channel, template version, delivery status, retry and failure. Communications should not block core order state changes.

SOURCE FILE: 04_LOGISTICS/01_COURIER_SYSTEM.md

Courier System

Activation

Courier is a capability on the same verified student identity.

Transport modes:

- walking,
- bicycle,
- motorcycle,
- car.

Eligibility/matching considers distance, package size/weight, route, time and transport mode.

Courier work loop

Go Online → eligible job card → map, distance, package class, payout → accept/decline → navigate to pickup → verify/scan custody → route to buyer/hub → OTP/scan handover → complete → earning state updates.

Verification

Walking: low-friction radius/availability policy.

Bicycle: transport declaration and capability constraints.

Motorcycle/Car: licence + vehicle details/evidence according to approved production verification policy.

Safety/privacy

Courier should receive only the data required for the current delivery. Cross-campus/transfer capability is separate from ordinary local courier capability.

SOURCE FILE: 04_LOGISTICS/02_CAMPUS_HUB_CUSTODY.md

Campus Hub and Package Custody

Campus Hub is the physical custody point inside the Vossie operating model.

Core workflows

- incoming package queue,
- scan/identify package,
- condition check,
- accept/reject/exception,
- custody state,
- ready for collection,
- courier handoff,
- outgoing transfer,
- returns.

QR

Package identifiers/QRs are generated by the backend/system and displayed/printed by the appropriate workflow. Scanning never directly grants a final state without server-side validation.

Custody chain

Every handoff records who/what role, where, timestamp, package/order and state transition.

Example:

Seller → Origin Hub → Local Courier/Transfer → Destination Hub → Courier/Buyer.

Disputes

Custody scans, condition records, OTP delivery and timestamps become evidence for Operations.

SOURCE FILE: 04_LOGISTICS/03_CROSS_CAMPUS_TRANSFERS.md

Cross-Campus Transfers

Hybrid model

Cross-campus is not a single student driving across provinces for every order.

Preferred model:

- campus hub network for pooled routes,
- third-party courier fallback for low-volume/long-distance cases.

Routing

For hub-to-hub movement, route through the **nearest viable** campuses, not merely nearest by straight-line distance. Viability includes route direction, schedule, capacity and operational availability.

Example concept:

Cape Town-area campus → next viable campus hub(s) → destination region/campus → local fulfilment.

User experience

Buyer sees one shipment and one tracking timeline, not many fragmented orders.

Operational model

Each leg has a manifest and custody scans. Intermediate hubs reconcile packages before forwarding.

Third-party fallback

If hub transfer is not economical (e.g. one parcel), offer approved external logistics such as parcel-point/direct courier where available. Third-party provider should be integrated through an adapter so the Vossie order still has a unified status view.

SOURCE FILE: 04_LOGISTICS/04_MAPS_GPS_INTEGRATIONS.md

Maps, GPS and Logistics Integrations

Prototype

Use real map UI where possible with simulated courier movement and predefined campuses/routes.

Buyer: - courier location/ETA where appropriate, - campus/hub status, - route milestones.

Courier: - current job, - pickup/drop-off route, - navigation context, - distance/ETA.

Hub/Transfers: - route/manifest visualization, - not unnecessary free-form personal tracking.

Operations: - network/exception visibility.

Incubation: - aggregated analytics, not individual courier GPS.

Production

Keep mapping/location provider behind an adapter. Background tracking must be evaluated on real iOS/Android devices. PWA behaviour is not assumed equivalent to native background location.

Privacy

Location access should be purpose-limited and retained only as long as operationally necessary.

SOURCE FILE: 05_OPERATIONS/01_SELLER_STUDIO_INCUBATION.md

Seller Studio and Incubation

Seller Studio

The deeper seller business-management experience includes:

- store profile,
- listing creation/editing,
- media,
- inventory,
- pricing/promotions,
- orders,
- earnings,
- payouts,
- seller analytics,
- timetable/availability,
- Reels/content analytics.

Timetable idea

Student uploads a timetable PDF → prototype extracts class times → student must review/confirm → availability is generated → student can manually block extra time even after class.

Do not treat extracted timetable data as authoritative without user confirmation. Avoid retaining the source PDF longer than needed where possible.

Incubation

Incubation is not simply seller admin. It sees aggregate/business-support intelligence:

- unmet searches,
- high-demand categories,
- stock-outs,
- seller growth,
- repeat demand,
- campus opportunities,
- business-health indicators,
- interventions and 30-day plans.

The distinction: seller sees their business; Incubation sees opportunity/support intelligence across authorized campus scope.

SOURCE FILE: 05__OPERATIONS/02__OPERATIONS__COMMAND__CENTER

Operations and Command Center

Vossie Operations should feel like a real operating console rather than a generic admin page.

Suggested navigation

Command Center | Users | Sellers | Couriers | Listings | Orders | Hubs | Transfers | Payments | Disputes | Support | Compliance | Incidents | Security | Audit | System

Command Center

Demo metrics may include clearly labelled sample data:

- campuses online,
- verified students,
- active sellers,
- couriers online,
- orders in progress,
- fulfilment success,
- attention-required cases,
- payment exceptions,
- custody mismatches,
- verification backlog,
- service/platform health.

Separation of duties

Use different grants for:

- moderation,
- support,
- finance,
- security,
- admin/configuration.

Do not create an unrestricted everyday “god mode” interface for all build-team users.

SOURCE FILE: 05_OPERATIONS/03_SUPPORT_COMPLIANCE_SECURITY

Support Tickets, Compliance and Security Operations

Support / disputes as cases

Each issue becomes a structured case containing:

- ticket/case ID,
- related order/payment/package,
- buyer/seller/courier/hub,
- timeline,
- evidence,
- messages,
- internal notes,
- assigned operator,
- priority/SLA,
- resolution.

Listing compliance

Possible state flow:

Draft → automated/basic checks → Published

or

Draft → Flagged → Human Review → Approved/Rejected.

Security operations

The admin security view can resemble a lightweight Vossie SOC dashboard, but it is not a claim of building a full SIEM product.

Useful events:

- failed auth patterns,
- suspicious sessions,
- role changes,
- payout changes,
- privileged actions,
- moderation evidence access,
- webhook/payment failures,
- repeated scan/custody anomalies.

Keep security logs separate from ordinary product analytics and never dump secrets into the UI.

SOURCE FILE: 06_UIUX_BRAND/01_BRAND_IDENTITY.md

Brand Identity

Brand direction

Vossie is soft, approachable, dependable and enterprising.

Core palette

- Warm Ivory #FAF8F2 — primary surface
- Vossie Navy #29475B — logo, copy, primary action

- Powder Blue #AFCBDE — selections/campaign panels
- Slate Blue #567487 — supporting information
- Pale Sage #DDE5D9 — restrained accent

Typography

Nimbus Sans is the supporting typeface in the approved brand profile. Use the approved custom logo artwork rather than retyping the wordmark.

Starting UI tokens

- Type scale: 32 / 24 / 20 / 16 / 14 px
- Spacing: 4 / 8 / 12 / 16 / 24 / 32 px
- Fields: ~12 px radius
- Cards: ~20 px radius
- Touch targets: ~48×48 px minimum goal

Logo rules

Preserve clear space and proportions. Avoid stretching, shadows, outlines, arbitrary recolouring and busy-photo placement.

Copy style

Friendly, direct, useful. Explain what happened and what the user can do next.

SOURCE FILE: 06_UIUX_BRAND/02_DESIGN_LANGUAGE_PLATO_UBI

Design Language: Plato-Inspired Visuals, Uber-Inspired Function

Plato-inspired visual literacy

Use as inspiration, not as copied proprietary design:

- airy layouts,
- gentle colour panels,
- warm surfaces,
- restrained typography,
- rich product photography,
- minimal visual noise,
- soft premium feeling.

Uber-inspired interaction literacy

Again, use patterns not proprietary copying:

- clear destinations,
- contextual cards,
- strong primary actions,
- bottom navigation on mobile,
- sheets/drawers,
- live status,
- map-centric courier flow,
- clear task switching,
- concise activity history,
- progressive disclosure.

Vossie rule

Visuals must remain unmistakably Vossie. Brand Identity wins over references.

Performance/latency feeling

Microinteractions should feel immediate. Essential actions must never wait for decorative animation. Stable typography and predictable state changes are more important than flashy transitions.

SOURCE FILE: 06_UIUX_BRAND/03_FIGMA_PRODUCTION_SYSTEM.m

Figma Production System

Figma is the UI/UX source of truth for the design phase.

Recommended file/page structure

00 Cover/Product Map 01 Brand & Foundations 02 Variables/Tokens 03 Core Components 04 Marketplace Components 05 Logistics Components 06 Operations Components 07 Access & Shared 08 Buyer 09 Seller 10 Courier 11 Campus Hub 12 Transfers 13 Incubation 14 Operations/Admin 15 Extensions 16 Responsive States 17 Motion & Interaction 18 Prototype Journeys 19 Developer Handoff

Build order

Brand/tokens → primitives → shared patterns → role patterns → priority journeys → full screen families → responsive states → motion specs → clickable prototype → developer handoff.

Component discipline

Use Figma Variables, Styles, Components, Variants and Auto Layout. Avoid meaningless frame names.

Examples:

- Marketplace/ListingCard/Food/Default
- Marketplace/Reel/ProductRail
- Courier/JobCard/Available
- Hub/PackageCard/AwaitingScan
- Operations/IncidentCard/Critical

States

Default, focus, pressed, hover where relevant, loading, disabled, success, error, offline, empty, retry, expired, denied, pending/review.

Handoff

Map Figma variables → code tokens; components → React components; variants → component states; frames → routes; responsive variants → container/breakpoint behaviour; motion specs → Motion.dev/CSS.

Automated Figma writing is an accelerator only when the connected plugin/account supports it; do not assume write access exists.

SOURCE FILE: 06_UIUX_BRAND/04_MOTION_ANIMATION_3D.md

Motion, Animation and Optional 3D

Principle

Functional motion first. Optional 3D second. Motion explains cause/effect and state.

Three layers

1. Presses, forms, navigation and recovery.
2. Matching, status, tracking and scanning.
3. Optional parcel preview/icon depth/campus illustration.

Core experiences must work without layer 3.

Existing motion inventory

The project defined ~30 motion contracts including:

- logo reveal,
- account-step transitions,
- button/submit feedback,
- main navigation,
- workspace switch,
- search continuity,
- sheets/filters,
- product-detail continuity,
- basket/quantity,
- seller matching,
- request arrival/acceptance,
- money-state transition,
- order milestones,
- courier interpolation,
- scan recognition,
- handover,
- manifest reconciliation,
- availability/workload,
- listing publication,
- messaging/notifications,
- incubation panels/plans,
- operations queue updates,
- loading/offline/recovery.

Runtime

Core motion uses CSS/Motion.dev-style implementation. Motion.so or other generative motion is optional exported media only and cannot be a runtime dependency for a zero-budget prototype.

Accessibility

Respect reduced motion. Never make animation the only carrier of meaning.

SOURCE FILE: 06_UIUX_BRAND/05_RESPONSIVE_PWA_DEVICE_EXP

Responsive PWA and Device Experience

Prototype distribution

Primary demo strategy: one URL/QR code opens the same adaptive PWA across iOS, Android, Windows and macOS.

Do not maintain a separate “phone website” and “desktop website.” Use one shared application/data model with adaptive layouts.

Phone

- bottom navigation,
- full-screen Reels,

- touch-first commerce,
- camera/QR,
- courier map,
- sheets/drawers.

Tablet

- split/contextual views,
- wider cards/maps,
- light operational use.

Desktop

- sidebars,
- multi-column layout,
- larger maps,
- seller studio,
- hub/incubation/admin tables and dashboards.

Native future

SwiftUI/native expansion is optional and should be justified by real requirements such as reliable background location, richer push or device integrations. The PWA remains the universal prototype base.

SOURCE FILE: 07_ENGINEERING/01_TECH_STACK.md

Technical Stack

Locked prototype foundation

Layer	Direction
Language	TypeScript
Web/PWA	Next.js + React
UI implementation	Tailwind CSS + Radix/shadcn-derived Vossie system
Backend	TypeScript modular monolith
Database	PostgreSQL
Prototype backend platform	Supabase
Data access	Drizzle + explicit SQL where needed
Validation	Zod
Authorization	Vossie capabilities/RBAC + server checks + RLS
Realtime	Supabase Realtime for selected flows
Testing	Vitest + Playwright
Repository	GitHub monorepo, pnpm + Turborepo
CI/CD	GitHub Actions
Prototype deploy	Vercel + Supabase

Candidates / optional layers

- Clerk as an auth candidate behind an identity-provider abstraction.
- SwiftUI as optional native iOS expansion.
- Production maps, payments, notifications and observability remain adapter/provider decisions.

Architecture rule

Start as a modular monolith with clear domains. Do not create microservices simply because the platform is complex.

Repository and Application Architecture

Monorepo principle

One GitHub repository is the durable source of truth for both IDEs and all agents.

Suggested areas:

- **apps/marketplace** — student-facing PWA
- **apps/operations** — staff/operations web app if separated at route/app level
- **packages/ui** — Vossie component system
- **packages/contracts** — API/events/validation shared types
- **packages/domain** — domain logic
- **packages/db** — schema/migrations/data access
- **packages/integrations** — payment/maps/identity/notifications adapters
- **packages/config** — lint/TS/design tokens etc.
- **.vossie/** — architecture, agent contracts, feature contracts, handoffs and status

Domain boundaries

Identity, Marketplace, Listings/Inventory, Requests, Orders, Payments/Ledger, Courier, Hub/Custody, Transfers, Messaging/Notifications, Disputes, Incubation, Operations/Audit.

Realtime rule

Realtime events improve speed but do not become source of truth. After reconnect, clients fetch authoritative current state from API/database.

SOURCE FILE: 07_ENGINEERING/03_AGENTIC_WORKFLOW.md

Agentic Workflow: ChatGPT, Cursor, VS Code and GitHub

Control plane

- **Architect (ChatGPT):** feature contracts, architecture, cross-system requirements.
- **Atlas (Cursor):** build lead; decomposes and integrates implementation work.
- **Guardian + Medic (Cursor):** workflow completeness and runtime/debug repair.
- **Sentinel (VS Code):** independent verification lead.
- **Watchtower + Resolver (VS Code):** verification audit and root-cause/recovery.
- **Harbor + GitHub Actions:** merge/release/deploy gates.
- **GitHub + .vossie/:** durable state.

End-to-end workflow

1. Requirement enters GitHub.
2. Architect creates/validates Feature Contract.
3. Atlas decomposes.
4. Branch/worktree isolation.
5. Specialists build.
6. Implementation tests.
7. Guardian audits completeness.
8. Medic runs/repairs ordinary failures.
9. Atlas integrates green work.
10. Atomic conventional commits pushed.
11. PR generated.
12. Sentinel dispatches independent checks.
13. Watchtower confirms current gates ran.

14. Resolver diagnoses failure; route back for repair.
15. Harbor merge gate.
16. CI/CD deploy.
17. Post-deploy smoke verification.
18. Status/evidence updated.

Boundary

Human approval remains required for destructive/high-risk production changes, credentials, security exceptions, ambiguous product choices and exhausted repair loops.

SOURCE FILE: 07__ENGINEERING/04__AGENT__ROSTER.md

Agent Roster and Responsibilities

Cursor build side

- **Atlas** — Lead Developer / build coordinator
- **Pixel** — Frontend/UI
- **Forge** — Backend/API/domain services
- **Schema** — Database/schema/migrations
- **Market** — Marketplace/order/request logic
- **Relay** — Logistics/courier/hub/transfers
- **Bridge** — Integrations/providers
- **Kinetic** — Motion/interactions
- **Probe** — Testing support
- **Scribe** — Documentation
- **Guardian** — workflow/dependency/completeness auditor
- **Medic** — build/runtime/debug repair loop

VS Code verification side

- **Sentinel** — verification controller
- **Lens** — code review
- **Aegis** — security review
- **Vector** — QA/E2E/release smoke
- **Pulse** — performance/accessibility
- **Vault** — data/schema/RLS review
- **Harbor** — Git/release gate
- **Watchtower** — verifies all required gates actually ran
- **Resolver** — deep failure analysis/recovery routing

Separation of duties

Atlas cannot approve/merge its own work. Sentinel cannot waive mandatory gates. Specialists cannot redefine locked architecture silently.

SOURCE FILE: 07__ENGINEERING/05__CICD__TESTING__SECURITY__GATES.md

CI/CD, Testing and Security Gates

Mandatory automated checks should include:

- clean install,
- formatting verification,
- lint,

- TypeScript/typecheck,
- unit tests,
- integration tests,
- production build,
- Playwright E2E,
- accessibility checks,
- dependency/security scanning,
- secret scanning,
- database migration validation,
- RLS/authorization validation,
- preview/staging smoke tests,
- post-deployment critical-path smoke tests.

Git rules

- scoped branches/worktrees,
- atomic conventional commits,
- PRs with feature/risk/test evidence,
- no silent direct push to main,
- protected merge after required checks,
- deployment evidence written back to GitHub/.vossie.

Reliability philosophy

Ordinary errors should enter bounded automated repair loops before reaching the user, but the system must never fake success or bypass a failed security/data gate.

SOURCE FILE: 07__ENGINEERING/06__INTEGRATION__ADAPTER__STRAT

Integration Adapter Strategy

All external dependencies should sit behind Vossie-owned interfaces/adapters.

Identity

Prototype simulated institutional sign-in; production Microsoft/Entra/Eduvos federation if approved.

Payments

Sandbox provider adapter now; production PSP later.

Maps/location

Provider-neutral map/location layer where possible.

Third-party courier

Provider adapter converts external tracking/events into Vossie shipment states.

Notifications

In-app is internal; push/email providers remain replaceable.

Storage/media

Prototype may use Supabase Storage; video-heavy Reels may later move to a more suitable media/CDN service.

Why

Adapters prevent the prototype from becoming permanently coupled to whichever free service was easiest during the Hack Jam.

SOURCE FILE: 08__BUSINESS__MARKETING/01__BUSINESS__MODEL__MAI

Business Model and Market Thesis

Business thesis

Vossie creates value across demand, supply, fulfilment and institutional support.

- Buyers: relevant campus supply, trust context, fulfilment visibility.
- Sellers: verified demand, storefront, orders, analytics, content discovery.
- Couriers: local earning opportunities with structured jobs.
- Eduvos/Incubation: visibility into entrepreneurship and unmet demand.

Monetization order

Do not optimize monetization before density and trust.

Pilot currently uses seller/courier commissions as hypotheses. Future levers may include:

- transaction/service fees,
- delivery economics,
- seller tools for power sellers,
- promoted discovery,
- institution/programme value,
- approved partner services.

Metrics

Track marketplace and logistics economics separately:

- GMV,
 - average order value,
 - take rate,
 - payment cost,
 - refund/loss rate,
 - delivery revenue,
 - courier payout,
 - support cost,
 - contribution margin,
 - search failure/request rate,
 - repeat purchase,
 - active sellers,
 - fulfilment success.
-

SOURCE FILE: 08__BUSINESS__MARKETING/02__MARKETING__GTM.md

Marketing and Go-to-Market

Positioning

“The campus marketplace that connects everyday needs with student opportunity.”

Brand campaign direction

Core campaign system developed around **GOOD THINGS. BRIGHTER CAMPUS.**

Message territories:

- What do you need?
- Made on campus.
- Sell here.
- Can't find it? Request it.
- See it. Tap it. Get it.
- Going that way?
- Good things. Closer.
- Your side hustle has a home.
- Buy. Sell. Move. Build.

Channels

- campus posters,
- street-pole/outdoor where permitted,
- tracked QR codes,
- Reels/social creatives,
- campus activations,
- seller/courier recruitment,
- student ambassador strategy,
- digital paid media later if budget exists.

Launch principle

Build recognition first, then explain feature depth. Lead with student outcome, not technical architecture.

SOURCE FILE: 08_BUSINESS_MARKETING/03_TEAM_ROLES.md

EduHackers Team Roles

Current roles captured in the project:

- **Oratile Modise — UI/UX Designer**
- **Babalo Kali — Developer & Research Lead**
- **Malindi Mpumelelo — Product Lead**
- **Danny Mohale — Project Manager & Pitch Lead**
- **Babalo Kali + Danny Mohale — Business Analysts & Marketing Leads**

Film/pitch mapping

- Malindi: fragmented campus needs become coherent product structure.
 - Oratile: product structure becomes design system, wireframes and refined UI.
 - Babalo: research becomes architecture, engineering, agents, testing and deployment.
 - Danny: workstreams converge through project management, then Danny leads the live product pitch.
 - Babalo + Danny: business analysis and marketing turn platform signals into positioning and growth narrative.
-

Cinematic Pitch Blueprint

Overall format

Two major acts plus finale.

Act I — ~4-minute cinematic origin/team film

- black frame,
- Vossie dot bounces,
- dot lands as the dot above the “i”,
- Vossie wordmark resolves,
- push through dot into illustrated campus economy,
- fragmented demand/supply channels become visible,
- Product/Design/Engineering/PM/Business contributions are introduced,
- full app is intentionally teased but not fully demonstrated.

Act II — ~15–20-minute Danny-led live/cueable product demonstration

Danny becomes Pitch Lead and the real Vossie prototype becomes the stage.

Suggested chapters:

REVEAL → BUYER → REELS → CHECKOUT → REQUEST → SELLER → COURIER → CROSS_CAMPUS → OPERATIONS → INCUBATION → RECAP → FINALE.

Act II should be presenter-controlled rather than one long fixed film. Each critical live path needs deterministic demo data, a reset procedure and prerecorded fallback.

Recurring visual motif

The Vossie dot is reused as identity, student/node, map pin, request, package, system event and transition.

Production architecture

Use modular sequences/comps rather than one giant 20+ minute timeline. After Effects is one possible production environment when actually available; alternate tools may be used if necessary.

Work Execution Handoff for Cinematic Production

When using ChatGPT Work for Series 4, provide:

- Developer Series 4 blueprint,
- approved Brand Identity,
- approved Figma files/links,
- working prototype,
- optional team portraits/campus references/audio/media.

Source precedence

Brand Identity controls branding. Approved Figma/prototype controls product UI. Series 4 controls narrative, shot structure and motion intent.

Execution rule

First audit assets and available tools. Create a production manifest mapping shot/cue IDs to assets, dependencies, app/tool, output and status. Then build sequence by sequence.

Do not claim Illustrator/After Effects/Figma write access unless the Work environment actually exposes those capabilities.

Production groups

- 01_IDENT
- 02_CAMPUS
- 03_TEAM_PRODUCT
- 04_TEAM_DESIGN
- 05_TEAM_DEV
- 06_TEAM_PM_BIZ
- 07_PITCH_HANDOFF
- 08_DEMO_OVERLAYS
- 09_LOGISTICS
- 10_OPERATIONS
- 11_FINALE
- 00_MASTER

QA

Brand fidelity, UI fidelity, missing media, fonts, composition dependencies, motion timing, legibility, audio sync, render errors, demo reset and offline/stage fallback.

SOURCE FILE: 10_REFERENCE/01_GLOSSARY.md

Glossary

- **Vossie Marketplace:** student-facing App 1.
 - **Vossie Operations:** App 2 for hub/incubation/admin/operations.
 - **Buyer:** base verified student marketplace capability.
 - **Seller:** activated marketplace capability for selling/business operation.
 - **Courier:** activated student delivery capability.
 - **Campus Hub:** physical custody/collection/transfer node.
 - **Transfer:** hub-to-hub movement across campus network.
 - **Incubation:** entrepreneurship/demand intelligence workspace.
 - **Request:** buyer-created demand object sent to eligible sellers.
 - **Reel:** short seller commerce video tied to listings.
 - **Custody:** accountable possession state of a package.
 - **OTP delivery proof:** recipient code proving final handover.
 - **PSP:** Payment Service Provider.
 - **Ledger:** append-only/reference-based money-movement record.
 - **RLS:** Row Level Security.
 - **RBAC:** Role-Based Access Control; Vossie also uses capabilities/resource scope.
 - **PWA:** Progressive Web App; universal prototype distribution route.
 - **Feature Contract:** engineering handoff describing requirement, scope, risks and acceptance.
 - **Atlas:** Cursor build lead agent.
 - **Sentinel:** VS Code verification lead agent.
 - **Guardian/Medic:** Cursor reliability agents.
 - **Watchtower/Resolver:** VS Code verification reliability agents.
-

SOURCE FILE: 10_REFERENCE/02_DECISION_TIMELINE.md

Decision Timeline

Early concept

- Eduvos student marketplace.
- Plato-inspired soft visual language and Uber-inspired interaction patterns.
- Buyer/Seller/Courier plus separate staff/incubation concept.

Architecture expansion

- Two application families locked.
- Verified identity and progressive roles.
- Campus hubs, QR custody, courier and cross-campus transfer architecture.
- 177-screen UI/UX atlas and larger developer handoff produced.

EduHackers 2.0 research session

- Pilot money model: 7% seller / 5% courier hypotheses.
- Pending/Secured/Available payout state model.
- QR custody + OTP delivery + 24-hour issue window.
- Eduvos/Microsoft SSO target with Authenticator MFA.
- Hybrid hub-to-hub + third-party cross-campus model.
- Buyer Requests confirmed.
- Dynamic inventory/category attributes confirmed.
- Food trust/compliance approach confirmed.
- Receipts/notifications/email event system confirmed.
- Reels confirmed as first-class marketplace tab.
- Courier transport modes and progressive verification confirmed.

Developer Series

- Series 1: adaptive PWA, stack and multi-agent architecture.
- Series 2: Figma/UI/UX/motion production.
- Series 3: agentic SDLC with Cursor/VS Code/GitHub gates.
- Series 4: cinematic pitch and Work handoff.

Current phase

Set up clean development/design environments and transfer project context into durable Markdown + repo/Figma sources rather than relying on chat history.

SOURCE FILE: 10_REFERENCE/03_SOURCE_FILES_INDEX.md

Canonical Source Files Index

Important original artifacts that should be retained alongside this Markdown pack when possible:

Strategy / architecture

- Vossie_Platform_Strategy_Differentiation_System_Blueprint_v12.pdf
- Vossie_Implementation_Specification.pdf
- Vossie_Developer_Architecture_and_Build_Guide.pdf
- EduHackers_Full_Developer_System_Architecture_Research_Dossier.pdf
- EduHackers_Developer_Build_Workflow_Technical_Research_Dossier.pdf

UI/UX / brand / motion

- Vossie_Full_UIUX_Page_Architecture.pdf
- Vossie_177_Screen_Map.pdf
- Vossie_Marketplace_Brand_UIUX_Profile.pptx
- Vossie_Motion_and_Interactive_3D_Showcase.pdf
- any approved Figma file/links and current live prototype.

Developer Series

- Vossie_Developer_Series_1_v3.pdf
- Vossie_Developer_Series_2_v1.pdf
- Vossie_Developer_Series_3_Draft_2_Final_Workflow.pdf
- Vossie_Developer_Series_4_Cinematic_Pitch_Production_Blueprint_v2.pdf

Business / marketing

- Vossie_Business_Analysis_and_Market_Research_2026.pdf
- Vossie_CMO_Marketing_Advertising_Playbook_2026.pdf
- Vossie_Visual_Advertising_Pack_2026.pdf

Rule

The Markdown pack summarizes and curates decisions; original assets remain valuable for visual detail, screen-level mapping and evidence.

SOURCE FILE: 10_REFERENCE/04_NEW_ACCOUNT_BOOTSTRAP_CHI

New ChatGPT Account Bootstrap Checklist

Use this when moving to a clean account.

Upload first

1. This complete Markdown ZIP or folder.
2. Brand Identity.
3. V12 Strategy Blueprint.
4. Full UI/UX Page Architecture + 177 Screen Map.
5. Developer Series 1–4.
6. Motion showcase.
7. Current Figma link/file.
8. Current working prototype link if still relevant.

First message

Paste the prompt from 00_START_HERE/02_MASTER_CONTEXT_TRANSFER_PROMPT.md.

Then ask the new account to

- read every Markdown file,
- output a one-page project-state summary,
- identify conflicts/open decisions,
- confirm source-precedence rules,
- avoid changing anything until it can reproduce the current two-app architecture, money flow, auth target, cross-campus model, Reels, Requests and agentic workflow accurately.

For Work mode

Tell Work which phase it is executing: Figma production, dev-environment setup, app implementation or cinematic production. Do not give a vague “continue Vossie” prompt in a brand-new workspace.

For repo use

Copy this knowledge base into `/docs/vossie/` or `.vossie/knowledge/` so Cursor and VS Code agents can read durable project context.

SOURCE FILE: README.md

Vossie Marketplace Knowledge Base

This folder is the curated Markdown knowledge base for Vossie Marketplace / EduHackers 2.0. It is intended to let a fresh ChatGPT account, a Work session, Cursor, VS Code, Figma, or a human teammate understand the project without relying on old chat memory.

What Vossie is

Vossie is not only a student buy-and-sell app. The current product thesis is a **verified campus commerce, logistics, operations and student-entrepreneurship platform** built around Eduvos students and campuses.

The platform has two application families sharing one identity and backend:

- **App 1 — Vossie Marketplace:** Buyer, Seller and Courier under one verified student identity.
- **App 2 — Vossie Operations:** Campus Hub, Transfers, Incubation, Support, Moderation, Finance, Compliance, Security, Admin and Platform Operations.

The current design universe contains **177 screens**, while the engineering handoff also defines a much larger implementation surface including database tables, APIs and delivery tickets. The prototype should prove complete vertical journeys rather than attempt to ship all 177 screens at once.

Recommended reading order for a fresh account

1. 00_START_HERE/01_PROJECT_CONTEXT.md
2. 00_START_HERE/02_MASTER_CONTEXT_TRANSFER_PROMPT.md
3. 00_START_HERE/03_SOURCE_OF_TRUTH_ORDER.md
4. 00_START_HERE/04_LOCKED_DECISIONS.md
5. 01_PRODUCT/01_PRODUCT_VISION_DIFFERENTIATION.md
6. 01_PRODUCT/02_TWO_APP_ARCHITECTURE_ROLES.md
7. 02_IDENTITY_SECURITY/01_AUTHENTICATION_EDUVOS_MFA.md
8. 03_COMMERCE/01_MONEY_MODEL_LEDGER.md
9. 04_LOGISTICS/03_CROSS_CAMPUS_TRANSFERS.md
10. 06_UIUX_BRAND/03_FIGMA_PRODUCTION_SYSTEM.md
11. 07_ENGINEERING/01_TECH_STACK.md
12. 07_ENGINEERING/03_AGENTIC_WORKFLOW.md
13. 09_PITCH_MEDIA/01_CINEMATIC_PITCH_BLUEPRINT.md

Important reality checks

- The current Vossie work is a **prototype/specification system**, not a production marketplace.
- Eduvos/Microsoft institutional SSO must be formally approved before production use. The prototype simulates the flow honestly.
- Live payment, payout, logistics and identity integrations require provider/institution credentials and approvals.
- Figma automation, After Effects access and third-party MCPs must only be treated as available when the current environment exposes them.
- Zero-budget is a hard constraint for the prototype; paid services must be optional, replaceable or simulated.

Folder map

- 00_START_HERE — migration/context transfer
- 01_PRODUCT — product model, roles, journeys, requests, Reels
- 02_IDENTITY_SECURITY — authentication, role verification, security
- 03_COMMERCE — money, payments, disputes, inventory, food, communications
- 04_LOGISTICS — courier, hub custody, transfers, maps
- 05_OPERATIONS — seller studio, incubation, admin, support/security operations
- 06_UIUX_BRAND — brand, Plato/Uber design language, Figma, motion, responsive behaviour
- 07_ENGINEERING — stack, repo architecture, agentic workflow, CI/CD, integrations
- 08_BUSINESS_MARKETING — market thesis, monetization, GTM, team roles
- 09_PITCH_MEDIA — cinematic pitch and Work handoff
- 10_REFERENCE — glossary, decision timeline, source index, fresh-account checklist